

网络流(EK + Dinic)

主讲：黄凤林

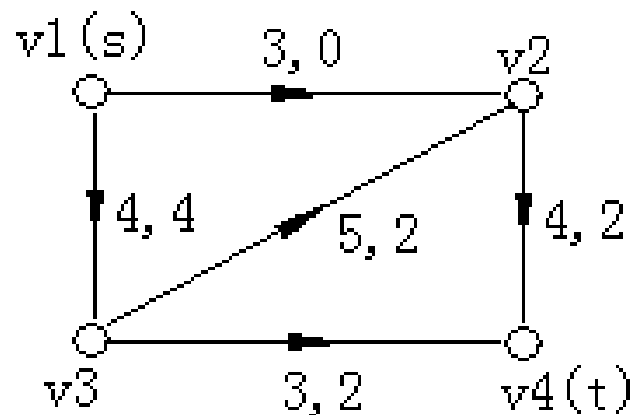
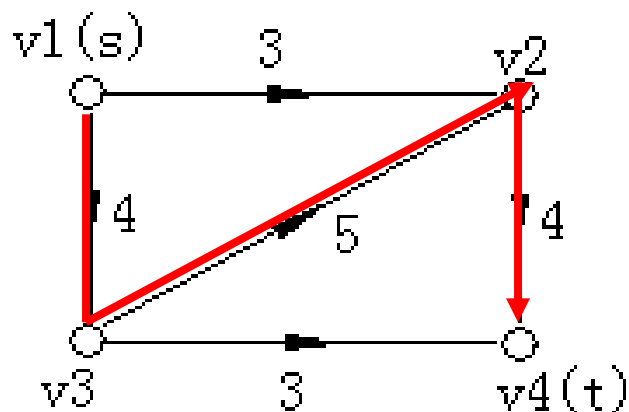
问题模型

问题描述:

如图所示，是联结某产品地 v_1 和销售地 v_4 的交通网，每一弧 (v_i, v_j) 代表从 v_i 到 v_j 的运输线，产品经这条弧由 v_i 输送到 v_j ，弧旁的数表示这条运输线的最大通过能力。

产品经过交通网从 v_1 到 v_4 。

现在要求制定一个运输方案使从 v_1 到 v_4 的产品数量最多。

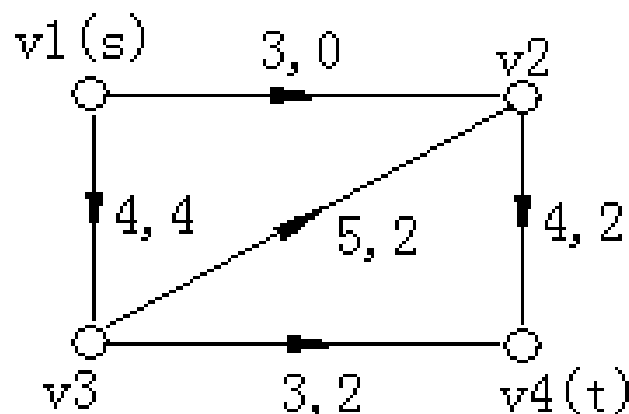
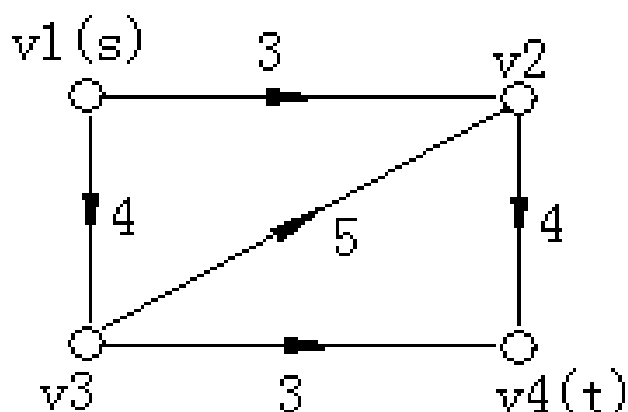


网络与网络流

网络：给一个有向图 $N=(V, E)$ ，在 V 中指定一点，称为源点(记为 v_s)，和另一点，称为汇点(记为 v_t)，其余的点叫中间点，对于 E 中每条弧 (v_i, v_j) 都对应一个正整数 $c(v_i, v_j) \geq 0$ (或简写成 c_{ij})，称为 f 的容量，则赋权有向图 $N=(V, E, c, v_s, v_t)$ 称为一个网络。

如图5-1所给出的一个赋权有向图 N 就是一个网络，指定 v_1 是源点， v_4 为**汇点**，**弧旁的数字为 c_{ij}** 。

网络流：所谓网络上的流，是指定义在弧集合 E 上一个函数 $f=\{f(v_i, v_j)\}$ ，并称 $f(v_i, v_j)$ 为弧 (v_i, v_j) 上的流量(下面简记为 f_{ij})。如图5-2所示的网络 N ，弧上两个数，第一个数表示容量 c_{ij} ，第二个数表示流量 f_{ij} 。



可行流与最大流

在运输网络的实际问题中，我们可以看出，对于流有两个显然的要求：

一是**每个弧上的流量不能超过**该弧的**最大通过能力**(即弧的容量)。

二是**中间点的流量为0**，源点的净流出量和汇点的净流入量必相等且为这个方案的总输送量。因此有：

(1)**容量约束**： $0 \leq f_{ij} \leq c_{ij}$, $(v_i, v_j) \in E$;

(2)**守恒条件**：对于中间点：流入量=流出量；对于源点与汇点：源点的净流出量 $v_s(f)$ =汇点的净流入量 $(-v_t(f))$ 的流 f ，称为网络 N 上的可行流，并将源点 s 的净流量称为流 f 的流值 $v(f)$ 。

网络 N 中**流值最大的流 f^* 称为 N 的最大流**。

→ 可增广路径

所谓可增广路径，是指这条路径上的流可以修改，通过修改，使得整个网络的流值增大。

设 f 是一个可行流， P 是从源点 s 到汇点 t 的一条路，若 p 满足下列条件：

(1)在 p 上的所有前向弧($v_i \rightarrow v_j$)都是非饱和弧，即
 $0 \leq f_{ij} < c_{ij}$;

(2)在 p 上的所有后向弧($v_i \leftarrow v_j$)都是非零弧，即
 $0 < f_{ij} \leq c_{ij}$;

则称 p 为(关于可行流 f 的)一条**可增广路径**。

→ 最大流定理

✧ 最大流定理:

当且**仅当不存在关于 f^* 的增广路径**，**可行流 f^* 为最大流**。

✧ 算法思想:

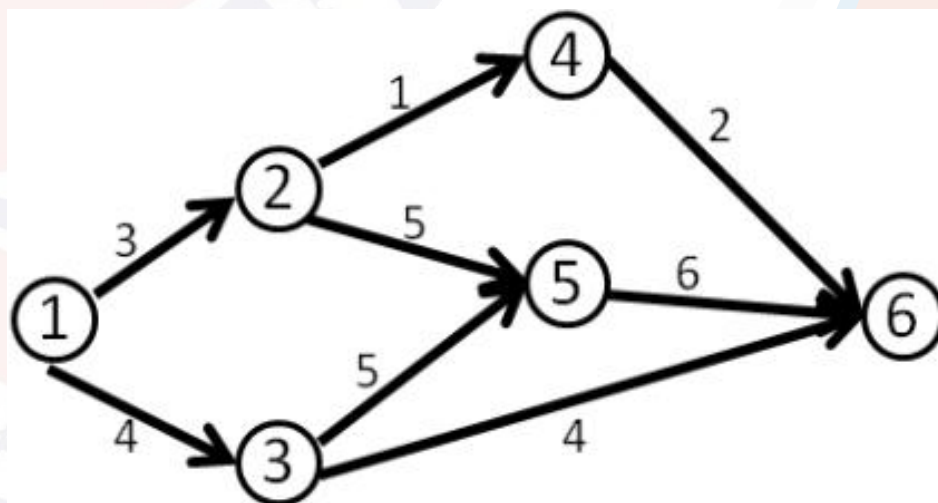
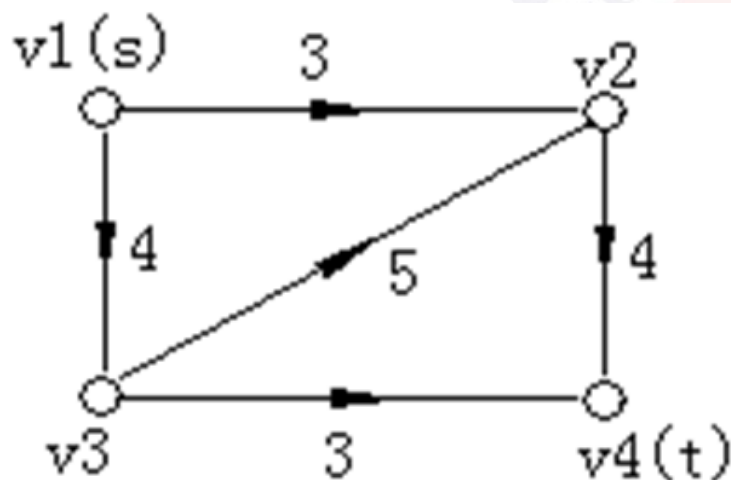
最大流问题实际上是求一可行流 $\{f_{ij}\}$ ，使得 vf 达到最大。

若给了一个可行流 f ，只要判断 N 中有无关于 f 的增广路径，如果有增广路径，改进 f ，得到一个流量增大的新的可行流；如果没有增广路径，则得到最大流。

→ 剩余网络(残量网络)

为了更方便算法的实现，一般根据原网络定义一个残量网络。其中 $r(u,v)$ 为残量网络的容量， $r(u,v) = c(u,v) - f(u,v)$

通俗地讲：就是对于某一条边(也称弧),还能再有多少流量经过。



➔ 增广路算法(EdmondsKarp)

✧ 寻找增广路过程

- 1.从源点出发，BFS找一条到汇点的增广路径。
- 2.如果找到，反向更新残留网络。
- 3.重复第1、2步，直到没有找到，算法结束。

EK时间复杂度： $O(nm^2)$



EK参考代码

```
const int maxn=1000+5;
const int inf=0x7f7f7f7f;
int map[maxn][maxn];
int n,m;
bool visit[maxn]; // 标记该点是否已经访问
int pre[maxn]; // 记录路径前驱
```

```
int main()
{
    int u,v,w;
    int s,t;
    scanf("%d%d",&n,&m);
    for(int i=1;i<=m;i++)
    {
        scanf("%d%d%d",&u,&v,&w);
        map[u][v]=w; // 有向图
    }
    scanf("%d%d",&s,&t);
    maxflow(s,t);
    return 0;
}
```

```
void maxflow(int s,int t)
{
    int inc=0; // 增加值
    int sumflow=0; // 总的最大流
    while(!(inc=bfs(s,t))) // 找到一条增广路
    {
        sumflow+=inc;
        int k=t; // 反向修改残留网络
        while(pre[k]!=0)
        {
            map[k][pre[k]]+=inc;
            map[pre[k]][k]-=inc;
            // printf("%d-->%d\n",k,pre[k]);
            k=pre[k];
        }
        // printf("%d \n",mincap);
    }
    printf("%d\n",sumflow);
}
```



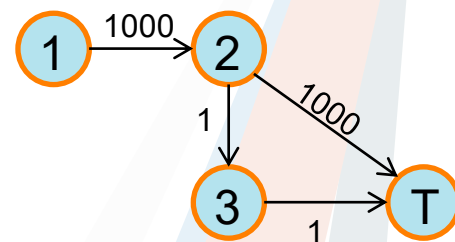
EK参考代码

```
int bfs(int s,int t)//找增广路
{
    queue<int> q;
    memset(visit,0,sizeof(visit));
    memset(pre,0,sizeof(pre));
    visit[s]=true;
    q.push(s);
    int mincap=inf;
    pre[s]=0;
    while(!q.empty())
    {
        int u=q.front(); q.pop();
        for(int i=1;i<=n;i++)
        {
            if(map[u][i]>0 && !visit[i])//有容量且未访问过
            {
                pre[i]=u;//记录路径
                mincap=min(mincap,map[u][i]);
                if(u==t) return mincap;//找到终点
                q.push(i); visit[i]=true;//入队, 标记
            }
        }
    }
    return 0;
}
```

EK的缺陷:

效率太低, 两大因素

- 1、每次更新流量太少
- 2、每次单路增广



Dinic算法

算法思想：

Dinic在找增广路的时候也是找的最短增广路, 与 EK 算法不同的是Dinic算法并不是每次bfs只找一个增广路, 他会首先通过一次bfs为所有点添加一个标号, 构成一个层次图, 然后在层次图中寻找增广路进行更新。

算法流程：

第一步：利用 BFS 对原来的图进行分层, 即对每个结点进行标号, 这个标号的含义是当前结点距离源点的最短距离(假设每条边的距离都为1), 注意：构建层次图的时候所走的**边的残余流量必须大于0**;

第二步：用DFS 寻找一条从源点到汇点的增广路, 注意: 此处寻找增广路的时候要按照层次图的顺序, 即如果将边(u,v)纳入这条增广路的话必须满足 **$dis[u] = dis[v] - 1$** , 其中 $dis[i]$ 为结点i的编号。找到一条路后要根据这条增广路径上的所有边的残余流量的最小值i更新所有边的残余流量(即正向弧-i, 反向弧+i);

第三步：重复步骤 2, 当找不到一条增广路的时候, 重复步骤 1, 重新建立层次图, 直到从源点不能到达汇点为止。

→ dinic参考代码

```
inline void addl(int u,int v,int w)
{
    edge[cnt].to=v;
    edge[cnt].w=w;
    edge[cnt].next=head[u];
    head[u]=cnt++;
}
```

```
void init()
{
    memset(head,-1,sizeof(head));
    n=read();m=read();x=read();
    int a,b,c;
    for(int i=0;i<m;i++){
        a=read();b=read();c=read();
        addl(a,b,c);addl(b,a,0);//加边和反向边
    }
}
```

```
int dini()//求最大流量
{
    int ans=0;
    while(bfs(1)){
        ans+=dfs(1,INF);//用INF是因为我们不知道此时的残量到底是多少
    }
    return ans;
}
```

→ dinic参考代码

```
bool bfs(int s){//bfs找深度, 建层次图, s为起点
    memset(dis, -1, sizeof(dis)); //注意! 不要忘了这句
    dis[s] = 0; //起点深度为0, 从源向汇bfs
    queue<int> q;
    q.push(s);
    while(!q.empty()){
        int u = q.front(); q.pop();
        for(int i = head[u]; i != -1; i = edge[i].next){
            int v = edge[i].to;
            if(dis[v] == -1 && edge[i].w)
                { //如果这个点还没有被bfs且这条边的流量不为0
                    dis[v] = dis[u] + 1;
                    q.push(v);
                }
        }
    }
    return dis[n] != -1; //若不可以到终点(起点)就返回false
}
```


→ dinic参考代码

```
int dfs(int u,int flo)//dfs就是求节点u及其子树在残量为flo时的最大增量
{
    if(u==n)return flo;//如果是汇，直接返回flo
    int detla=flo;//用一个变量来保存flo，以方便更新其剩余残量
    for(int i=head[u];i!=-1;i=edge[i].next)
    {
        int v=edge[i].to;
        if(dis[v]==(dis[u]+1) && (edge[i].w)>0)//如果满足深度条件且边可以走
        {
            int d=dfs(v,min(detla,edge[i].w));//向下递归
            //求v及其后续节点在残量为min(detla,edge[i].v)时的最大流量
            edge[i].w-=d;edge[i ^ 1].w+=d;//边的处理
            //注意，编号为i的边的反向边的编号为i^1,这是根据存边时的cnt决定的
            detla-=d;//将本节点的剩余残量值更新
            if(detla==0)break;//如果本节点已经没有残量可消耗了，就返回
        }
    }
    return flo-detla;//返回这个点已经被允许流过的流量
}
```

→ P1343 地震逃生

✧ 题目描述

汶川地震发生时，四川**中学正在上课，一看地震发生，老师们立刻带领 x 名学生逃跑，整个学校可以抽象地看成一个有向图，图中有 n 个点， m 条边。1号点为教室， n 号点为安全地带，每条边都只能容纳一定量的学生，超过楼就要倒塌，由于人数太多，校长决定让同学们分成几批逃生，只有第一批学生全部逃生完毕后，第二批学生才能从1号点出发逃生，现在请你帮校长算算，每批最多能运出多少个学生， x 名学生分几批才能运完。

✧ 输入格式

第一行3个整数 n, m, x ($x < 2^{31}, n \leq 200, m \leq 2000$)；以下 m 行，每行三个整数 a, b, c ($a \neq b, c > 0$) 描述一条边，分别代表从 a 点到 b 点有一条边，且可容纳 c 名学生。

✧ 输出格式

两个整数，分别表示每批最多能运出多少个学生， x 名学生分几批才能运完。如果无法到达目的地（ n 号点）则输出“Orz Ni Jinan Saint Cow!”



→ 练习题目

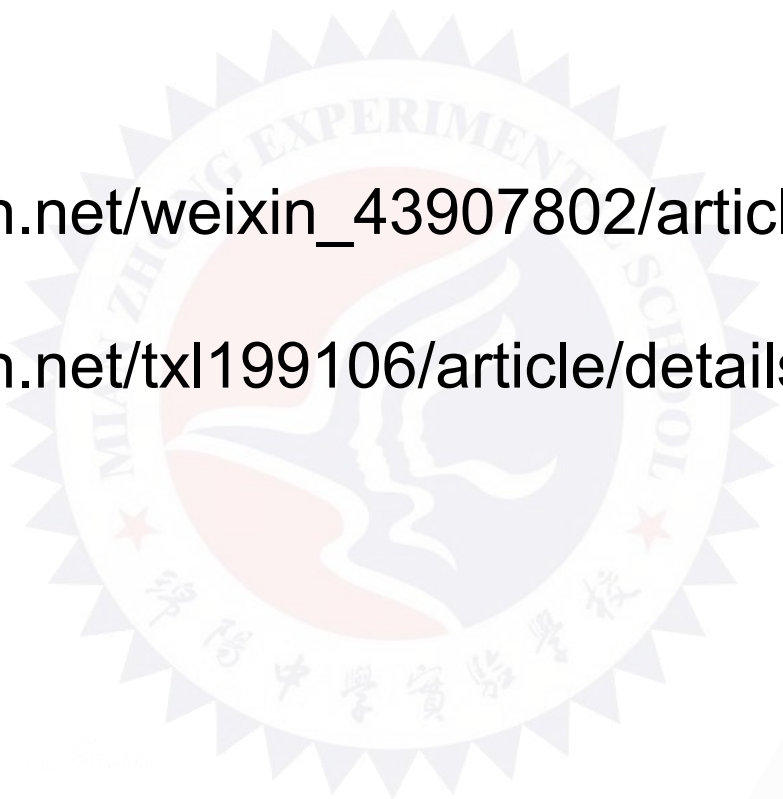
hdoj 1532

参考学习博客：

https://blog.csdn.net/weixin_43907802/article/details/84705855

提升学习博客：

<https://blog.csdn.net/txl199106/article/details/64441994>



The background features a central point from which several wide, colorful rays (in shades of blue, orange, red, and grey) radiate outwards. Faint binary code (0s and 1s) is scattered across the background, and a small line graph with two fluctuating lines is visible on the right side.

Thank you!